

# Recap tecnico delle modifiche alla pipeline topologica

## Object detection e ricostruzione topologica di diagrammi elettrici

### Nota per la tesi

Questo documento riassume le modifiche introdotte nella pipeline fino alla versione attuale, con particolare attenzione ai componenti critici emersi durante lo sviluppo: **operational amplifier**, **NPN transistor**, **MOSFET** e ai passi **04, 05, 06, 07 e 08**.

I riferimenti ai file sono riportati nel testo in modo da poter risalire facilmente all'implementazione.

## 1. Obiettivo generale della pipeline

La pipeline ha l'obiettivo di trasformare un diagramma elettrico 2D in una rappresentazione topologica strutturata, passando progressivamente da:

1. **rilevamento dei componenti**;
2. **assegnazione delle istanze**;
3. **stima dei terminali**;
4. **estrazione dei collegamenti (wire)**;
5. **costruzione delle net**;
6. **matching terminale-net**;
7. **esportazione del grafo**;
8. **visualizzazione finale del grafo**.

L'evoluzione della pipeline è stata guidata soprattutto da tre problemi pratici:

- la necessità di localizzare i terminali **non solo per lato**, ma anche **nel punto corretto lungo il lato**;
- la difficoltà di gestire componenti con struttura interna complessa, come **opamp**, **transistor NPN** e **MOSFET**;
- la necessità di rendere robusta la ricostruzione topologica anche in presenza di **testo**, **simboli interni**, **terminali impliciti** o **linee corte di alimentazione**.

## 2. Struttura generale della pipeline

### Passo 01 — Rilevamento dei componenti

File principale: `01_detect_components.py`

Il primo passo esegue la detection dei componenti tramite un modello **YOLO**, utilizzando:

- un insieme di classi ricavate da `class_terminals_v1.yaml` ;
- il filtro delle classi da rilevare;
- il salvataggio di un JSON per immagine;

- il salvataggio di una immagine di debug con i bounding box.

Ogni componente salvato nel JSON contiene almeno:

- `class_id`
- `class_name`
- `conf`
- `bbox`
- `symbol_type`
- `flag use_for_terminals`
- `flag use_for_masking`

Questo passaggio costituisce la base di tutta la pipeline: ogni fase successiva lavora sui bounding box prodotti qui.

## Passo 02 — Assegnazione delle istanze

File principale: `02_assign_instances.py`

Nel secondo passo i componenti rilevati vengono:

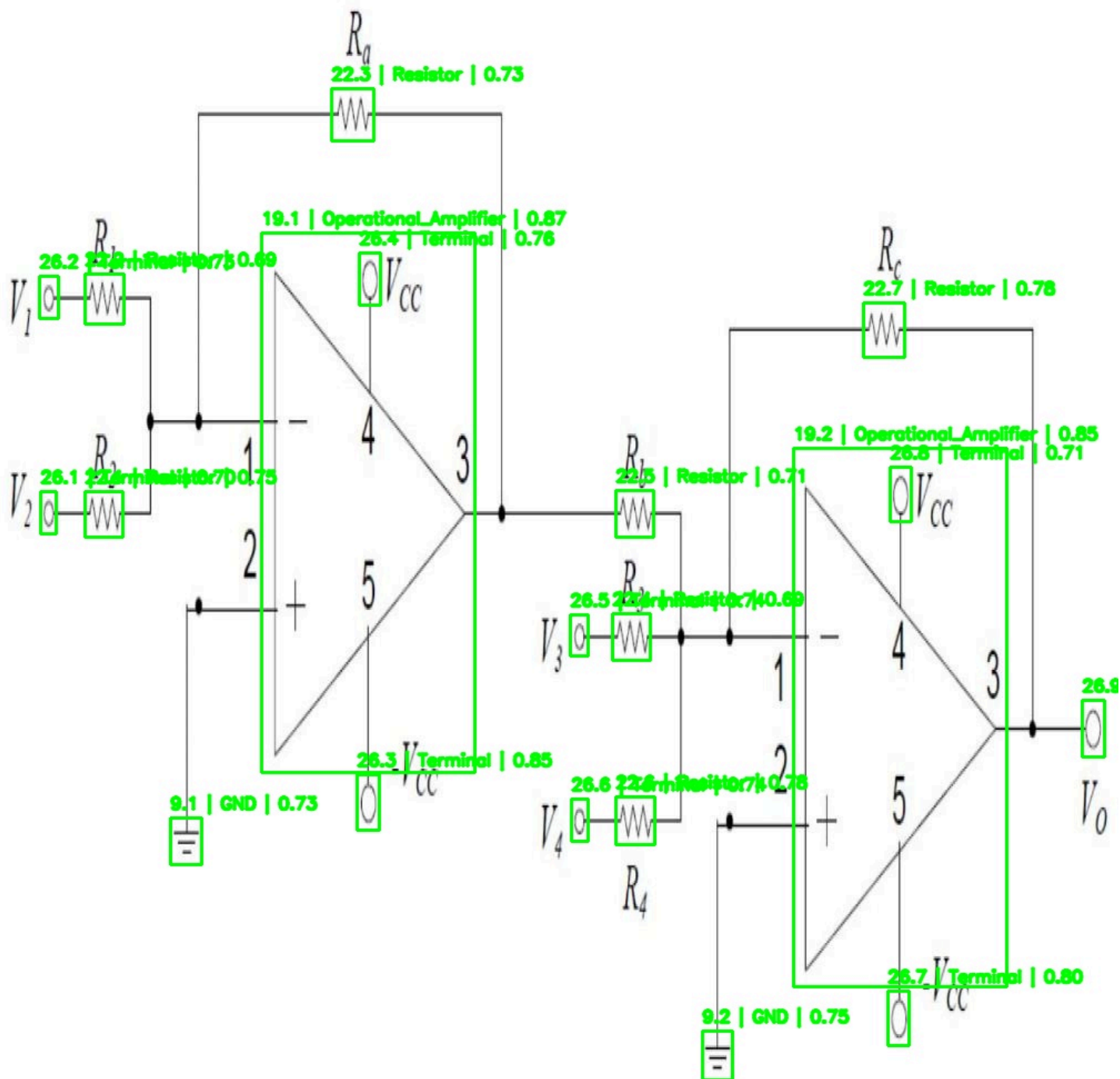
- raggruppati per `class_id` ;
- ordinati nello spazio;
- numerati in modo consistente;
- salvati con un identificativo `instance_id` del tipo `class_id.indice` .

L'ordinamento adottato è configurabile ( `xy` oppure `yx` ), ma nella versione attuale è usato **`xy`** , cioè:

- prima da sinistra verso destra,
- poi dall'alto verso il basso.

Questo passaggio è fondamentale perché:

- stabilizza l'identità delle istanze all'interno del diagramma;
- rende leggibile il debug;
- permette di costruire in seguito nodi di grafo con ID coerenti.



### 3. Passo 03 — Stima dei terminali

File principale: 03\_estimate\_terminals.py

Moduli principali coinvolti:

- geometry.py
- strategies\_opamp.py
- strategies\_terminal\_class.py
- strategies\_three\_terminal.py

Il passo 03 è quello che ha subito le modifiche più importanti.

Lo scopo è stimare, per ogni componente, non solo **quanti terminali ha**, ma anche:

- il **lato corretto**;
- la **posizione precisa del terminale**;
- eventuale **orientazione stimata** del simbolo;
- dati di debug utili per analizzare i casi difficili.

Le strategie principali restano:

- `fixed`
- `auto_by_aspect_ratio`
- `one_terminal_by_orientation`
- `two_terminal_by_connection_axis`
- `terminal_auto_one_or_two`

Tuttavia il comportamento è stato raffinato con logiche specifiche per le classi più problematiche.

### 3.1. Miglioramento generale della geometria dei terminali

File principale: `geometry.py`

In origine la geometria dei terminali era principalmente basata sul **centro geometrico del lato del bounding box**.

Questa soluzione è semplice ma insufficiente nei casi in cui:

- il terminale non sia centrato sul lato;
- il simbolo contenga testo o grafica interna che disturba;
- più terminali insistano su lati diversi ma richiedano una localizzazione più precisa.

Per questo è stata introdotta e consolidata una logica di **localizzazione “side peak”**:

- si individua il lato attivo del componente;
- si scansiona il lato lungo il suo asse naturale;
- si costruisce un profilo 1D di supporto del foreground;
- si seleziona il picco più plausibile, raggruppando i massimi in run consecutive;
- si usa il centro della run migliore come coordinata finale del terminale.

Questa idea è stata estesa in modo particolare ai componenti a 3 terminali e all'opamp.

Inoltre, la geometria è stata resa più robusta con:

- funzioni di clamp dei bounding box all'immagine;
- inferenza dell'orientazione da rapporto d'aspetto;
- sonde locali per capire se il terminale è realmente sostenuto da un wire esterno.

### 3.2. Classe `Terminal` : passaggio da logica rigida a logica evidence-based

File principale: `strategies_terminal_class.py`

La classe `Terminal` è stata trattata come caso speciale perché spesso compare come piccolo simbolo isolato, vicino a testo o simboli di alimentazione.

La filosofia adottata è diventata:

- **default = 1 terminale;**
- si passa a **2 terminali solo se l'evidenza è davvero forte;**
- si evita ogni forzatura derivante dal bordo immagine;
- si applica una soppressione locale del testo per non farsi influenzare da etichette come `V` , `Vcc` , `Vo` , ecc.

Le migliorie principali sono state:

1. **costruzione di un binary locale text-suppressed:**

si estrae una ROI attorno al bbox, si trovano le connected components e si mantengono solo quelle che intersecano davvero il bbox del terminale;

2. **shape prior leggero:**

il bbox può suggerire se il simbolo è più verticale o più orizzontale, ma questo bias viene applicato in modo leggero e non ai casi quasi quadrati;

3. **classificazione 1 terminale vs 2 terminali** basata su:

- score locali;
- score “far” più lontani dal bbox;
- confronto tra ipotesi mono-terminale e bi-terminale;

4. **tie-break con supporto passante:**

nei casi ambigui si misura quanto il terminale sembri una connessione reale che entra/esce dal simbolo, invece di essere solo testo o rumore grafico;

5. **validazione point-based:**

per i casi residui viene stimato il punto effettivo su ogni lato e si valuta il supporto direzionale verso l'esterno.

Il risultato è una stima molto più conservativa e robusta dei terminali di questa classe.

### 3.3. Componenti a 3 terminali: NPN e MOSFET

File principale: `strategies_three_terminal.py`

Supporto geometrico: `geometry.py`

I componenti a 3 terminali hanno richiesto un lavoro specifico, perché non basta capire “su quale lato” cade il terminale singolo: bisogna anche capire **dove localizzare i due terminali ortogonali** e quale orientazione sia realmente corretta.

#### 3.3.1. Geometria strutturata dei 3 terminali

In `geometry.py` è stata introdotta una geometria dedicata:

- il lato singolo viene trattato separatamente;
- i due terminali ortogonali vengono cercati nella regione corretta del bbox;
- viene introdotto il concetto di **pair bias** per capire se la coppia ortogonale stia effettivamente sul lato opposto oppure in configurazione specchiata;
- il punto finale non viene più fissato in modo rigido, ma cercato con `side_peak` in finestre mirate.

Questa parte è stata fondamentale per rendere più stabili i casi NPN e MOSFET.

### 3.3.2. MOSFET: validazione finale tramite terminal points

Per i MOSFET non ci si affida più solo a score generici di lato singolo.

La strategia è stata resa più forte introducendo:

- generazione dei tre terminali coerenti con ogni orientazione candidata;
- scoring diretto dei terminali finali stimati;
- confronto tra orientazioni candidate sulla base dei terminal points;
- supporto opzionale a score laterali del gate.

Di fatto, la scelta dell'orientazione non è più soltanto “quale lato sembra attivo”, ma “quale orientazione genera un set di terminali più coerente con i wire osservati”.

### 3.3.3. NPN transistor: prefilter d'asse e bonus per il lato base

Per gli NPN è stata introdotta una validazione finale analoga ai 3-terminali generici, ma con due miglioramenti mirati:

#### 1. **axis prefilter**

prima di testare tutte le orientazioni, si confrontano gli score lungo l'asse orizzontale e verticale, per evitare ribaltamenti errati dovuti a testo o grafica interna;

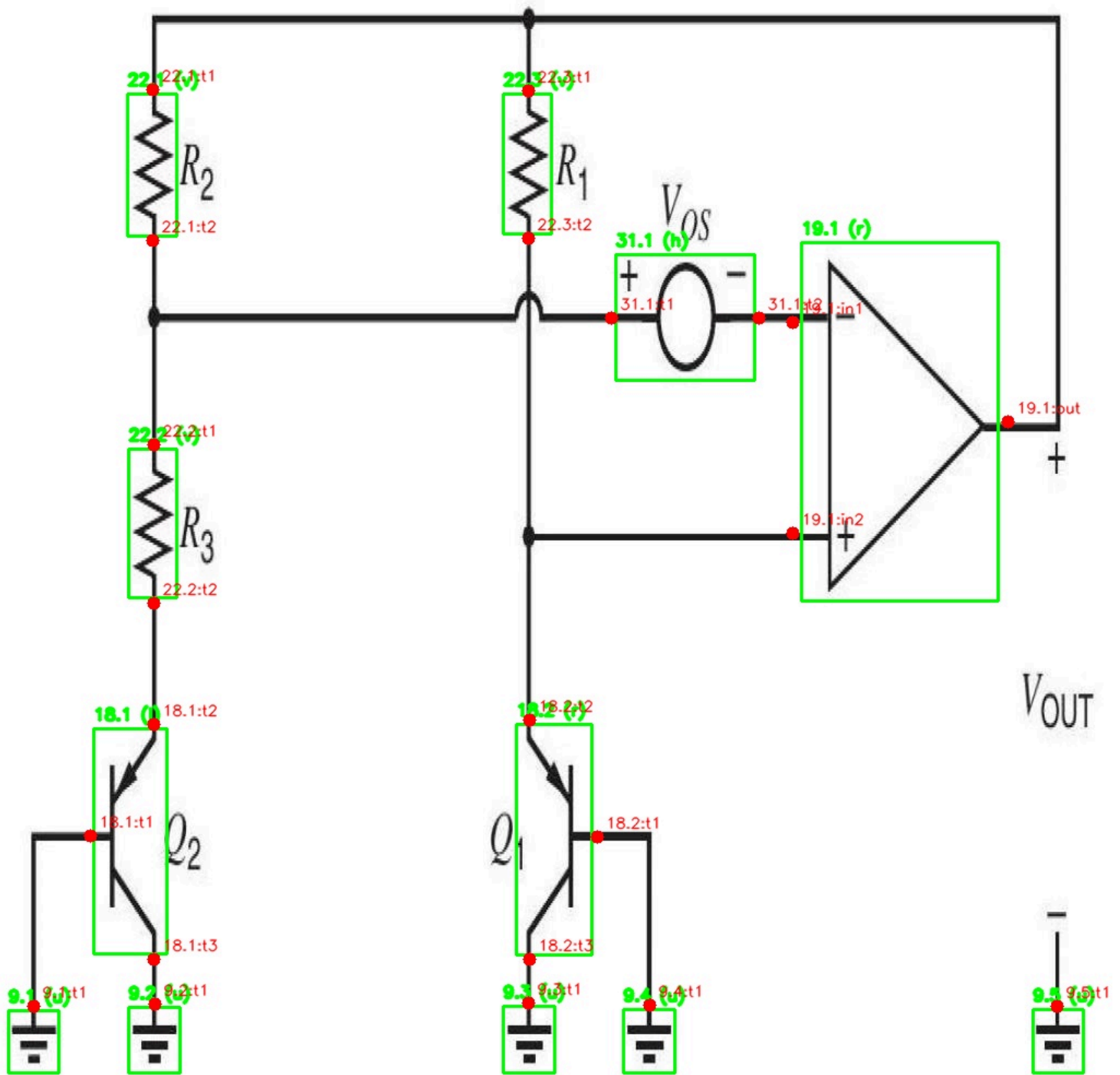
#### 2. **base-side bonus**

per la classe `NPN_Transistor` viene calcolato uno score dedicato ai lati compatibili con la base, e questo viene usato come bonus nella scelta tra orientazioni candidate.

Questa logica ha reso più stabile la distinzione tra casi `left/right` e casi ribaltati erroneamente `top/bottom`.

**Figura suggerita da inserire:**

[INSERIRE FIGURA 3: esempio di transistor NPN con terminali corretti dopo la validazione finale]



### 3.4. Operational amplifier: reset strategico e gestione dei pin ausiliari

File principali:

- strategies\_opamp.py
- geometry.py
- 03\_estimate\_terminals.py

L'opamp è stato il caso più complesso dell'intera pipeline, perché:

- contiene numeri e simboli interni ( + , - , 1 , 2 , 3 , 4 , 5 );

- può avere pin opzionali di alimentazione ( `aux1` , `aux2` );
- questi pin opzionali possono essere disegnati in modi diversi o anche essere impliciti;
- il punto corretto del pin di alimentazione non coincide sempre con il simbolo collegato sopra/sotto.

Per risolvere questi problemi è stato introdotto un **reset strategico** dell'opamp.

### 3.4.1. Orientazione stimata usando solo i terminali obbligatori

La prima scelta importante è stata questa:

- l'orientazione dell'opamp viene stimata usando **solo i tre terminali obbligatori**:
  - `in1`
  - `in2`
  - `out`

Per ognuna delle orientazioni candidate ( `right` , `left` , `top` , `bottom` ) si generano i terminali obbligatori e si misura il supporto direzionale.

L'output pesa leggermente di più degli input, così la stima finale favorisce l'orientazione che spiega meglio anche il ramo di uscita.

Questo riduce fortemente il rumore introdotto dai pin ausiliari.

### 3.4.2. Mandatory terminals con probe quasi tutta esterna al bbox

La localizzazione di `in1` , `in2` e `out` è stata resa molto più robusta:

- le probe sono strette;
- lavorano quasi completamente **fuori dal bbox**;
- il bordo del simbolo viene usato come conferma leggera;
- la grafica interna dell'opamp pesa pochissimo.

Questo ha permesso di evitare che numeri e simboli interni falsassero il punto terminale.

### 3.4.3. Rilevamento strutturale dei terminali opzionali `aux1` e `aux2`

Per i pin ausiliari è stata introdotta una strategia a più stadi:

#### 1. rilevamento del ramo verticale

si cerca una run verticale connessa davvero al lato superiore o inferiore del bbox, nella banda centrale dell'opamp;

#### 2. refine del punto sul lato obliquo

una volta capito che l'aux esiste, il punto viene riportato sul giunto interno con la diagonale del triangolo, invece di restare sul simbolo esterno eventualmente collegato;

#### 3. refine locale della x dello stelo verticale

si migliora l'allineamento dello stelo per evitare drift laterali;

#### 4. allineamento comune dei due aux

se `aux1` e `aux2` sono entrambi attivi e quasi allineati, si impone un asse `x` comune.

### 3.4.4. Snap del pin ausiliario superiore a un nearby terminal

Nel file `03_estimate_terminals.py` è stato aggiunto un post-processing esplicito:

- `snap_opamp_top_aux_to_nearby_terminal(updated_components, image_binary)`

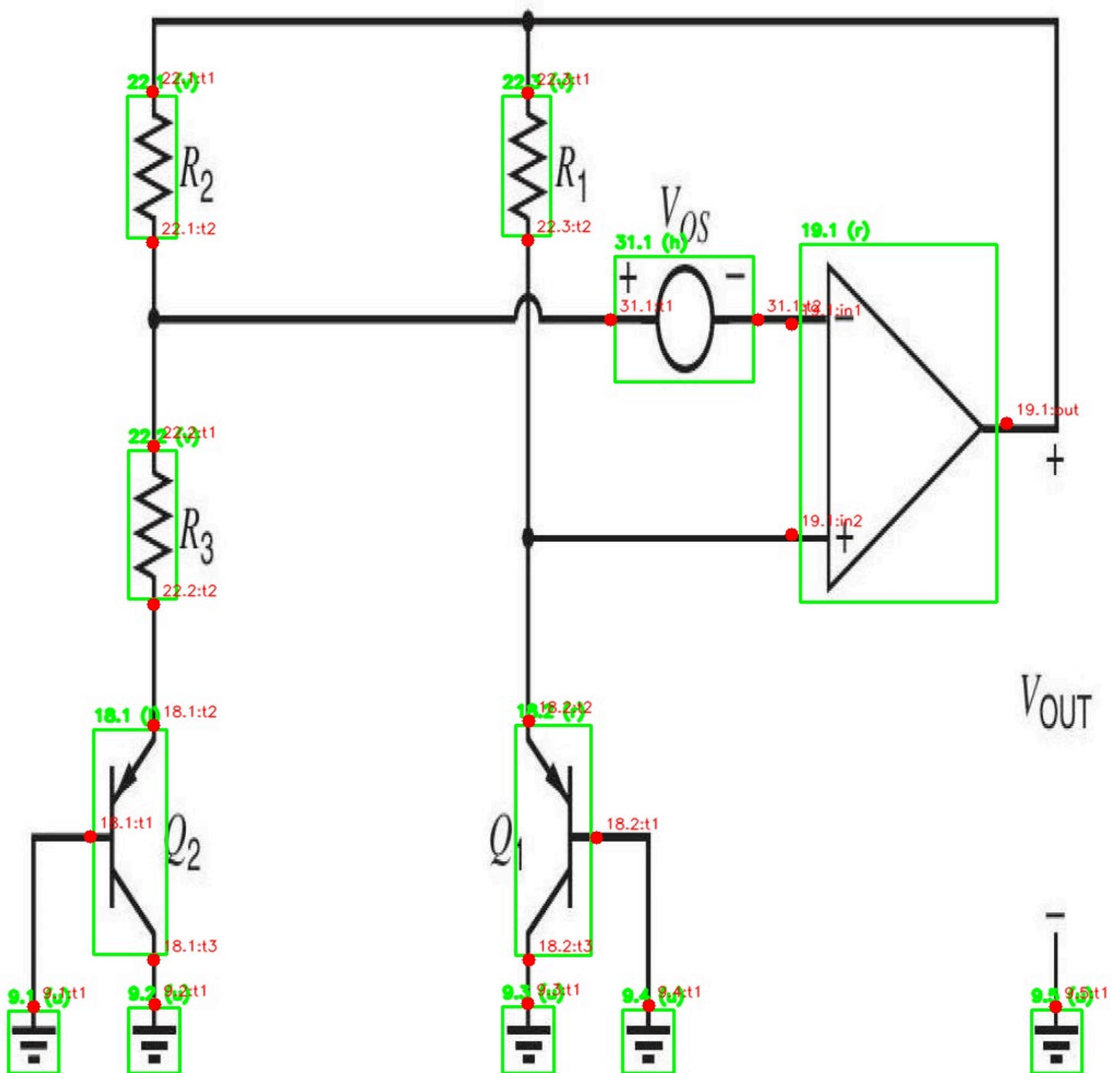
L'idea è questa:

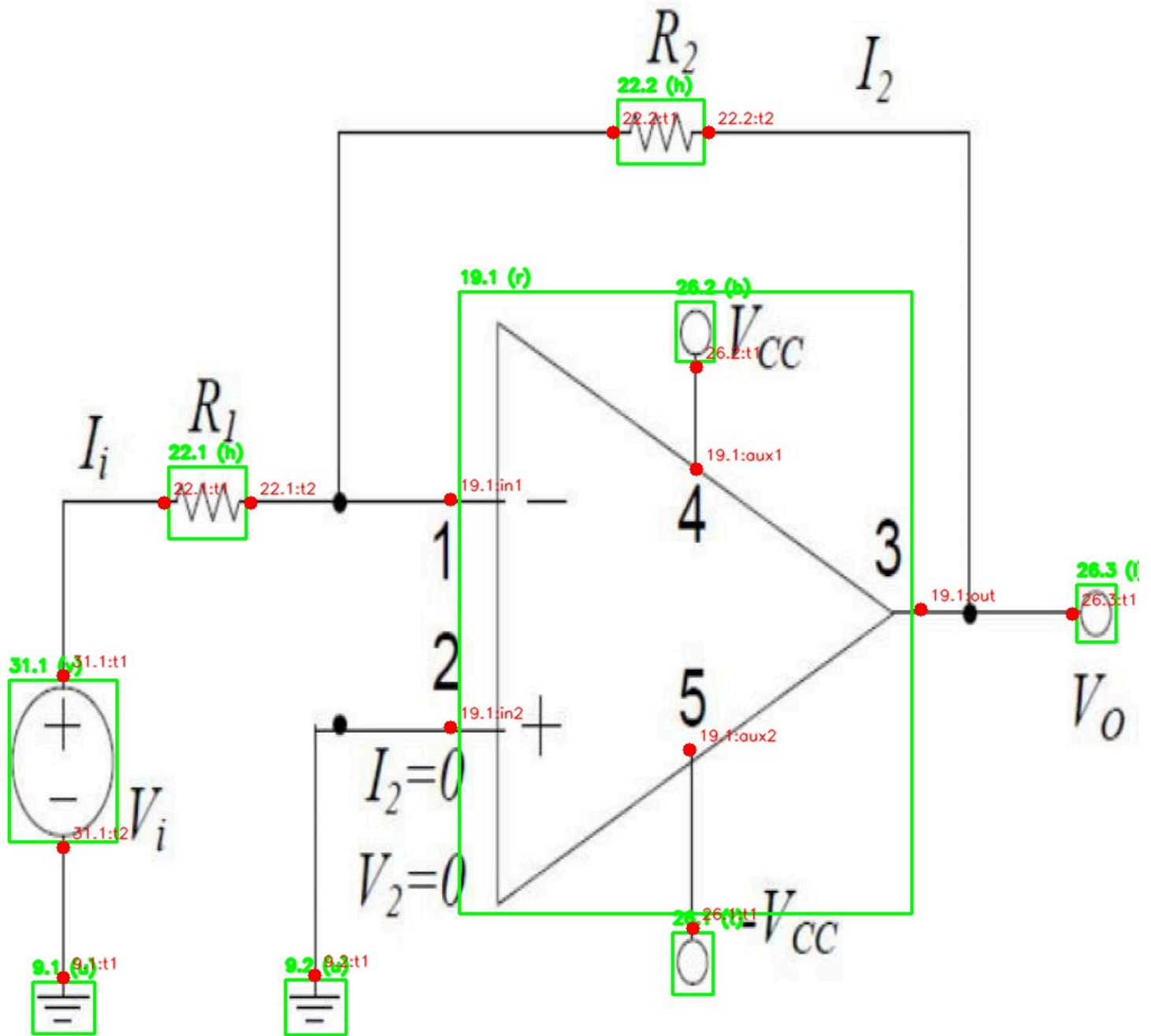
- se sopra l'opamp è presente un piccolo `Terminal` separato (tipicamente `Vcc` o `Vdd` ),



- l'asse  $x$  del pin aux1 viene corretto usando quel terminale vicino,
- ma il punto finale rimane il **giunto interno** sulla diagonale dell'opamp.

Questa modifica è stata importante per i casi in cui il punto dell'aux superiore tendeva a scivolare verso il numero interno 4 .





## 4. Passo 04 — Estrazione dei wire

File principale: 04\_extract\_wires.py

Il passo 04 costruisce l'immagine binaria dei wire mascherando i componenti, ma preservando localmente le zone dei terminali.

La pipeline del passo 04 è:

1. conversione in grayscale;
2. costruzione della maschera dei componenti;

3. carving delle **terminal keep zones**;
4. threshold;
5. morphological closing;
6. rimozione delle connected components troppo piccole;
7. skeletonization.

Le modifiche principali introdotte sono state:

## 4.1. Maschera dei componenti con shrink del bbox

I bbox dei componenti non vengono usati “pieni”, ma ridotti con `MASK_SHRINK_FACTOR`.  
Questo evita di cancellare troppo wire in prossimità del bordo del simbolo.

## 4.2. Terminal keep zones locali

Per ogni terminale si preservano due elementi:

- un **cerchio locale** attorno al punto terminale;
- una **capsula direzionata** coerente con il lato stimato.

Questa modifica è fondamentale perché un terminale stimato non cade sempre perfettamente sul wire: lasciare solo il punto non basta, mentre una piccola zona orientata rende la maschera molto più tollerante.

## 4.3. Parametri dedicati per gli aux dell'opamp

Gli auxiliary dell'opamp hanno keep zones dedicate:

- raggio minore;
- spessore minore;
- inward/outward diversi.

In questo modo si preservano i piccoli rami di alimentazione senza aprire troppo la maschera.

## 4.4. Small component filtering

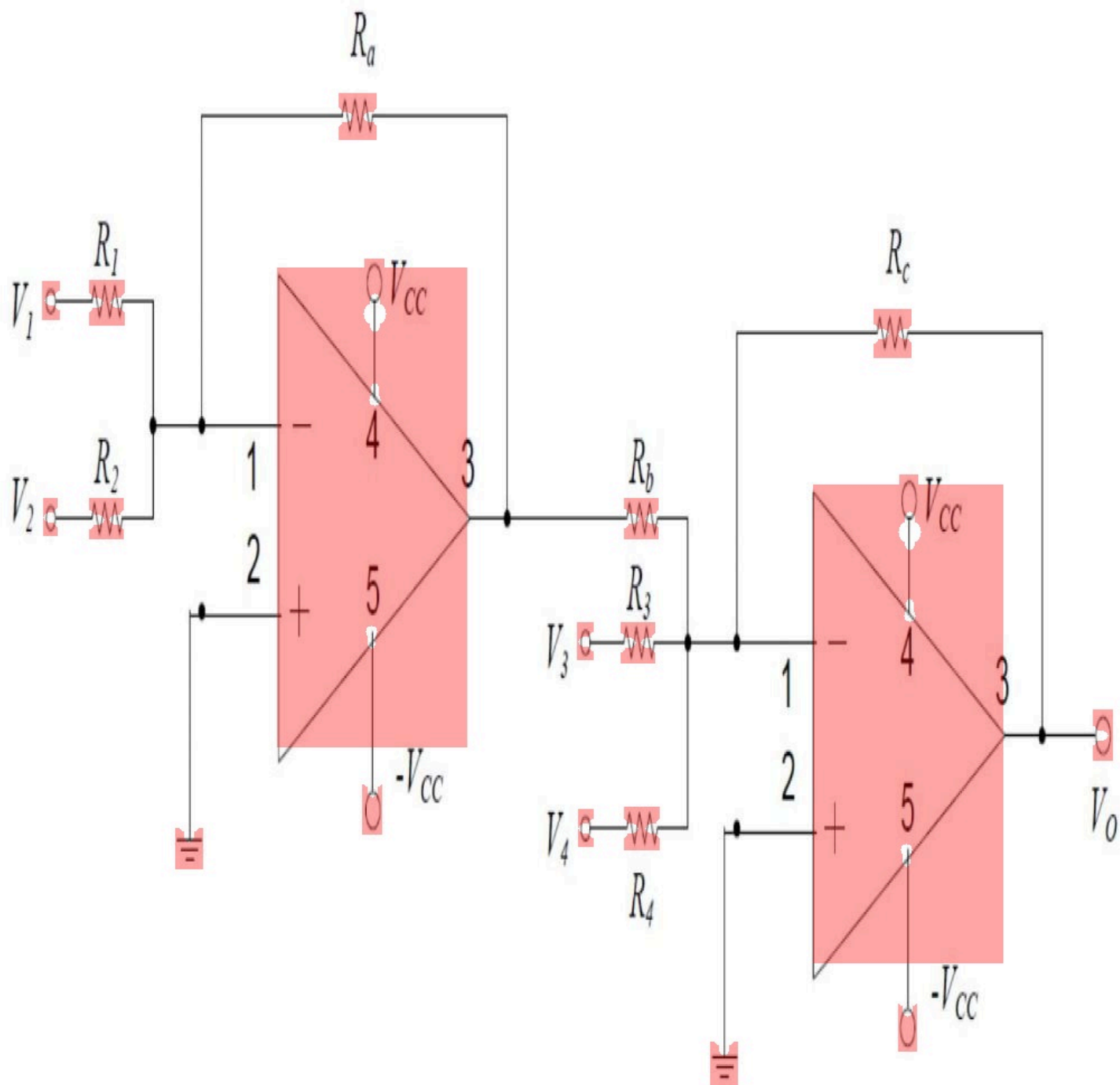
Prima della skeletonization viene applicato un filtro sulle connected components troppo piccole.  
Questo riduce rumore, residui di testo e frammenti inutili.

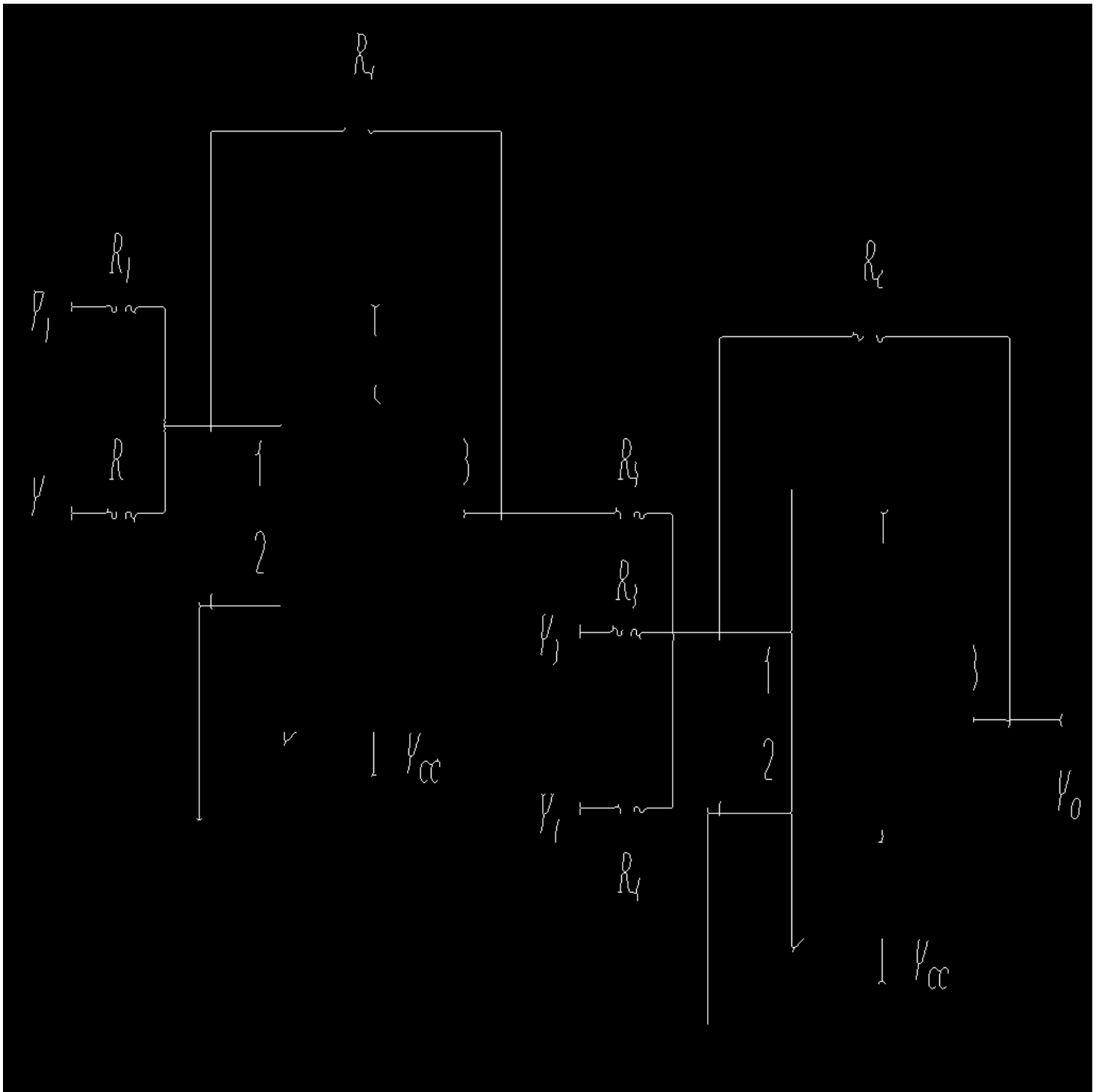
## 4.5. Output di debug molto più ricco

Il passo 04 salva:

- `component_mask`
- `mask_debug`
- `terminal_keep_debug`
- `masked_gray`
- `binary`
- `closed`
- `filtered`
- `skeleton`

Questo rende possibile analizzare in modo molto più fine dove si perda o si spezzi la struttura dei wire.





## 5. Passo 05 — Costruzione delle net

File principale: 05\_builds\_nets.py

Il passo 05 costruisce le net candidate a partire dallo skeleton del passo 04.

La pipeline è:

1. connected components dello skeleton;
2. matching locale terminale  $\rightarrow$  label;
3. costruzione delle net candidate;

4. filtraggio delle candidate;
5. rilabeling delle net mantenute;
6. salvataggio della label map e delle immagini di debug.

Questo passo è stato profondamente modificato, soprattutto per gestire i pin ausiliari dell'opamp.

## 5.1. Matching locale standard dei terminali

Per i terminali standard il matching viene eseguito con due stadi:

- finestra direzionale coerente col lato del terminale;
- fallback con finestra quadrata.

Per ogni terminale si memorizzano:

- label candidate;
- label primaria;
- snap point sullo skeleton;
- distanza di snap;
- finestra usata.

## 5.2. Matching dedicato per aux1 e aux2

Per i terminali ausiliari dell'opamp è stata introdotta una logica dedicata:

- corridoio verticale più ampio;
- scoring basato su reach verso l'esterno;
- penalizzazione del disallineamento in  $x$  ;
- penalizzazione dei pixel dal lato sbagliato;
- supporto a label preferite.

## 5.3. Ereditarietà dal terminale vicino

Se in fase 03 un aux è stato agganciato a un nearby terminal, nel passo 05 può:

- ereditare direttamente la label primaria del terminale vicino;
- evitare di creare una nuova net spuria.

Questa è una modifica molto importante per le alimentazioni esplicite con simbolo `Terminal` separato.

## 5.4. Gestione dei supply impliciti

Nel caso in cui nel corridoio dell'aux non si trovi un terminale reale, ma si osservi uno stelo compatibile con una connessione di alimentazione, il passo 05 può costruire una **implicit supply net**:

- viene identificata una label anchor;
- si fondono eventuali label consecutive appartenenti allo stesso ramo;
- la net risultante viene marcata come `is_implicit_supply = True` .

Questa logica serve per i casi in cui l'alimentazione è disegnata ma non è modellata con un terminale esplicito.

## 5.5. Merge di source labels in una net unica

Per le implicit supply o per i casi di merge controllato, le net possono salvare:

- source\_label
- merged\_source\_labels
- implicit\_reason
- implicit\_anchor\_terminal\_id

Questa informazione verrà poi propagata ai passi successivi.

## 5.6. Filtro delle net a singolo terminale

Il filtro delle candidate è stato raffinato:

- una net con un solo terminale deve avere una consistenza minima in pixel e span;
- eccezione: se quella net tocca solo terminali ausiliari dell'opamp, il filtro forte non viene applicato.

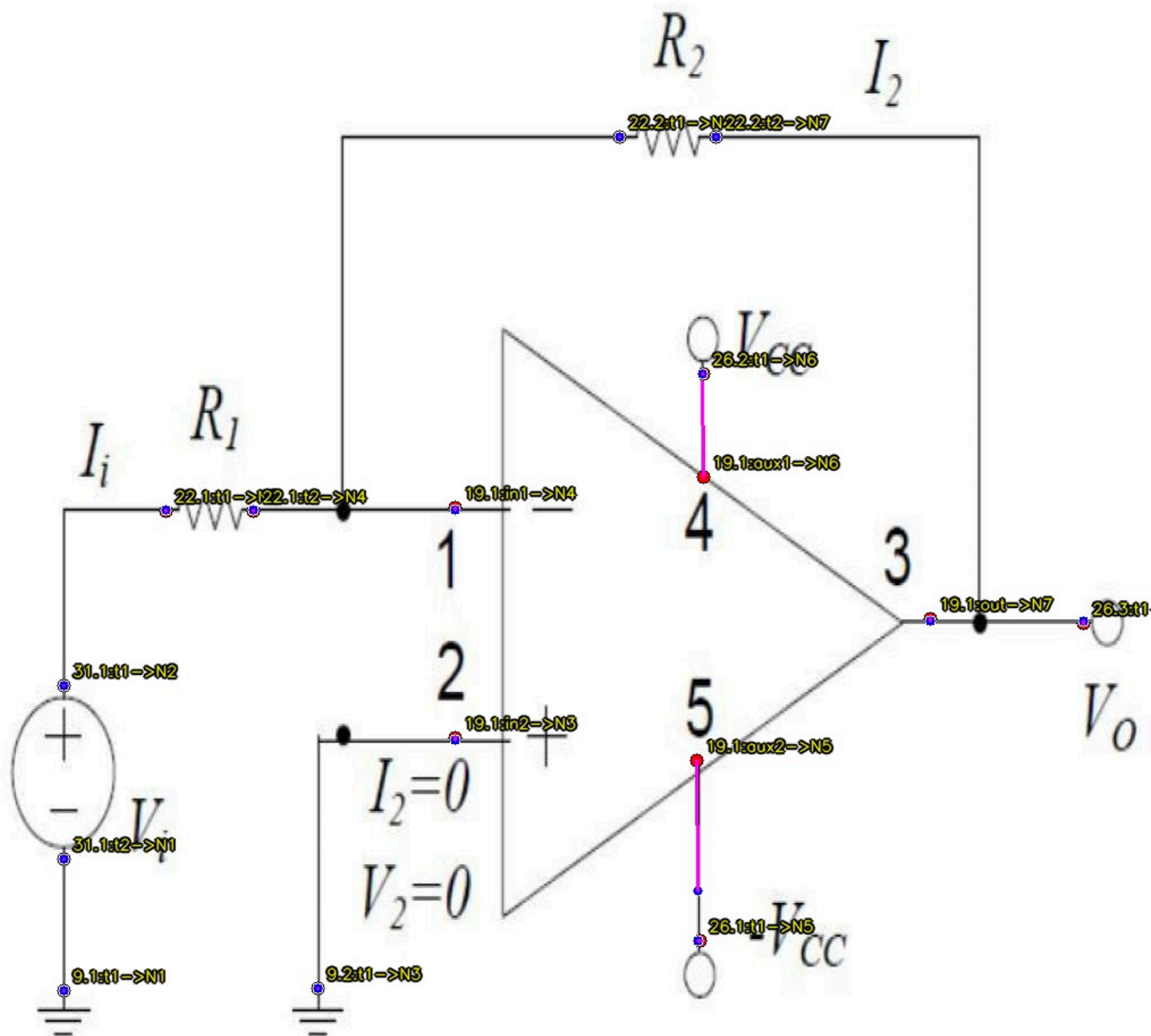
In questo modo non si eliminano alimentazioni corte ma valide.

## 5.7. Configurabilità del dataset

Il passo 05 supporta `PIPELINE_DATASET` da variabile d'ambiente, così la stessa implementazione può essere riutilizzata su varianti diverse della pipeline senza hardcodare i path.

**Figura suggerita da inserire:**

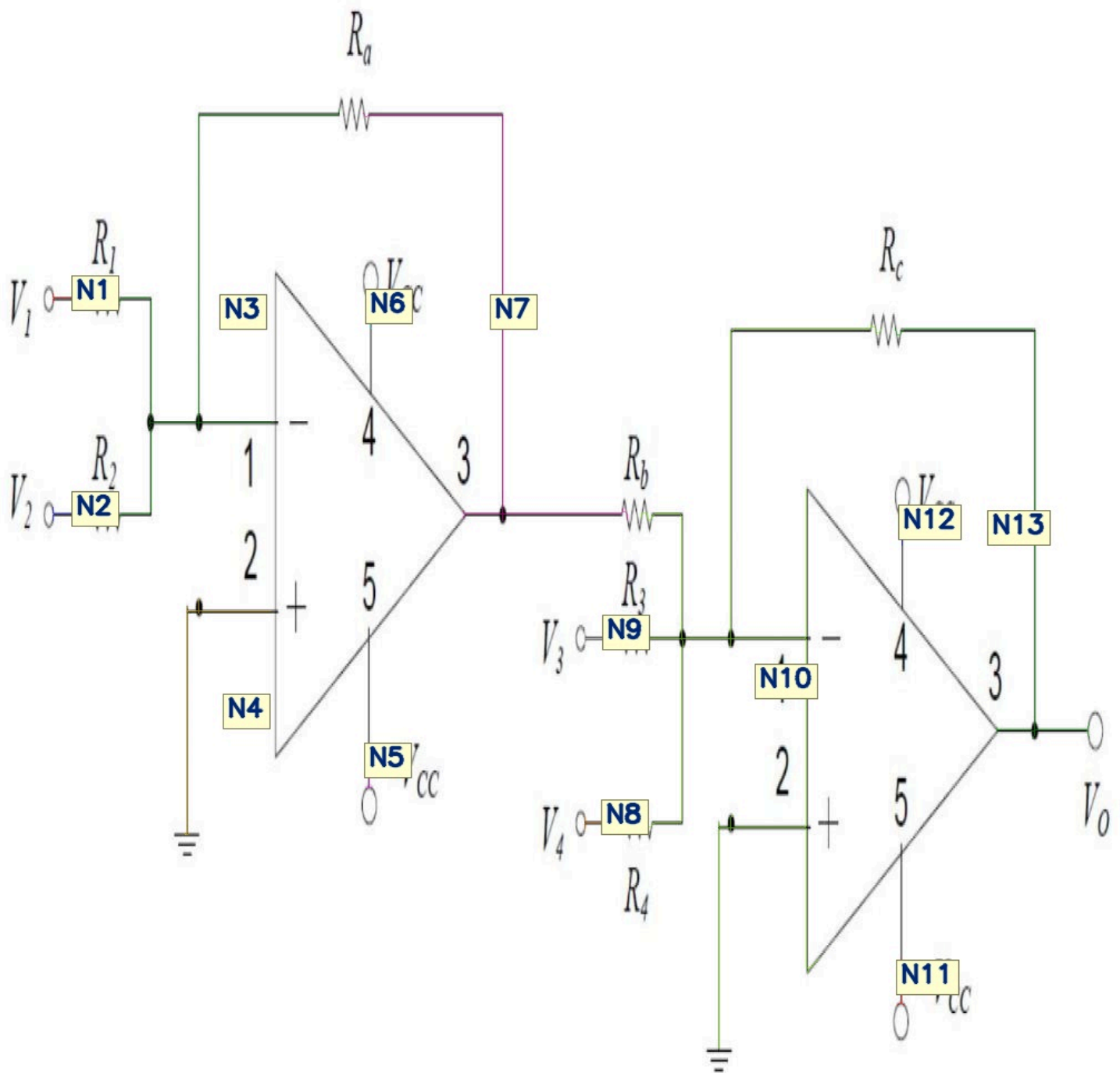
[INSERIRE FIGURA 9: terminal debug del passo 05 con snap point e net associate]



**Figura suggerita da inserire:**

[INSERIRE FIGURA 10: net overlay del passo 05 con etichette N1, N2, ...]





## 6. Passo 06 — Matching terminali → net finali

File principale: 06\_match\_terminals\_to\_nets.py

Il passo 06 assegna a ogni terminale una **net finale**, partendo dalla label map e dalle net costruite al passo 05.

La logica è stata resa molto più robusta e gerarchica.

### 6.1. Preferred net dal passo 05

Per ogni terminale si costruisce una **preferred net** usando:

- il `primary_label` salvato dal passo 05;
- la mappa `source_label -> net_index`.

Questo permette di non ricominciare da zero nel passo 06, ma di usare il passo 05 come prior informativo.

## 6.2. Sequenza di search stages

Se la preferred net non basta, il terminale viene testato con una sequenza di stadi:

1. `directional_primary`
2. `circle_primary`
3. `directional_fallback`
4. `circle_fallback`

L'ordine riflette l'idea che la ricerca direzionale sia la più affidabile, mentre il cerchio sia un fallback.

## 6.3. Matcher verticale dedicato per gli aux dell'opamp

Per `aux1` e `aux2` esiste uno stage specializzato:

- `opamp_aux_vertical`

Questo stage cerca in una capsula verticale e privilegia:

- continuità outward;
- vicinanza in `x`;
- supporto sufficiente in numero di pixel;
- eventuale bonus verso la preferred net.

## 6.4. Preferred-from-05 window stage

Per gli aux dell'opamp può essere usato anche uno stage ancora più mirato:

- `preferred_from_05_window`

In questo caso si riutilizza direttamente la finestra salvata nel passo 05 per tentare il rematch sulla net preferita.

## 6.5. Stati di match più espressivi

Il passo 06 non si limita più a dire "matched / unmatched", ma distingue stati come:

- `matched_preferred`
- `matched_implicit_supply`
- `matched_single`
- `matched_nearest`
- `unmatched`

Questa informazione è molto utile sia per la diagnosi sia per l'esportazione finale del grafo.

## 6.6. Confidence e warnings

Ogni match finale viene classificato con:

- `match_confidence`

- `match_warnings`
- `is_suspicious_match`

La confidence finale attuale è essenzialmente:

- `ok`
- `unmatched`

Ma l'informazione di warning aggiunge contesto, ad esempio:

- uso di fallback search;
- match senza preferred label;
- distanza troppo grande;
- uso del matcher verticale dedicato.

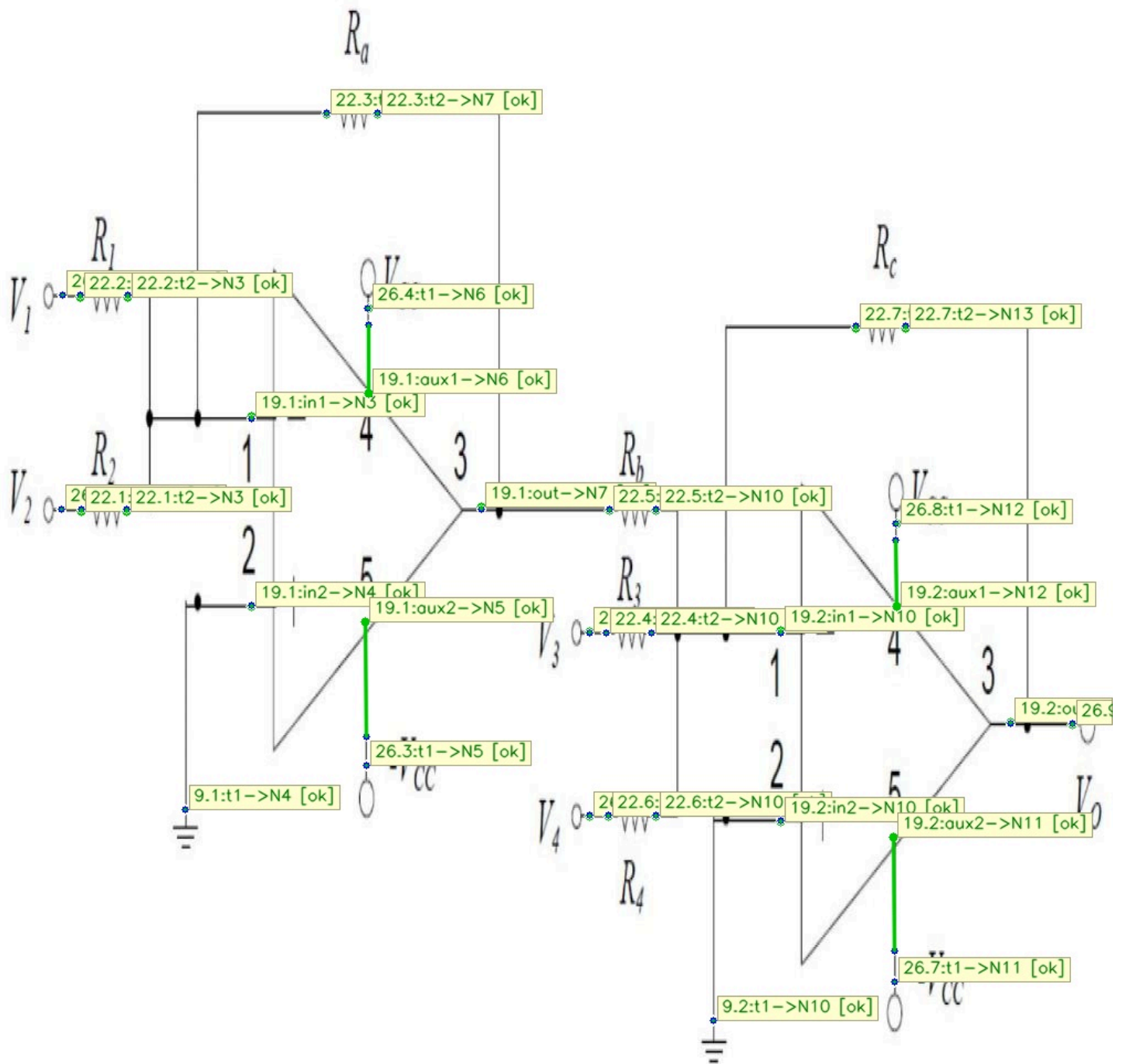
## **6.7. Aggiornamento coerente di componenti, terminals e connections**

Il passo 06 aggiorna:

- la lista globale dei terminali;
- i terminali annidati nei componenti;
- la lista delle connessioni terminale-net.

In output vengono anche salvate statistiche come:

- numero di terminali matched/unmatched;
- numero di match sospetti;
- lista degli ID sospetti.



## 7. Passo 07 — Esportazione del grafo

File principale: 07\_export\_graph.py

Il passo 07 trasforma il risultato topologico in una struttura a grafo esplicita.

### 7.1. Modello del grafo

Il modello adottato è:

- Diagram -> HAS\_COMPONENT -> Component
- Diagram -> HAS\_NET -> Net
- Component -> HAS\_TERMINAL -> Terminal
- Terminal -> CONNECTED\_TO -> Net

Questa struttura rende il risultato compatibile con analisi successive, interrogazioni e visualizzazioni.

## 7.2. Nodi e archi con ID univoci batch-wide

Ogni nodo viene costruito con ID univoci del tipo:

- diagram:<id>
- component:<diagram\_id>:<instance\_id>
- terminal:<diagram\_id>:<terminal\_id>
- net:<diagram\_id>:<net\_id>

Questo evita collisioni tra diagrammi diversi.

## 7.3. Propagazione delle informazioni diagnostiche

Nel grafo non vengono salvati solo i collegamenti, ma anche molte informazioni utili:

- match\_status
- match\_confidence
- match\_warnings
- is\_suspicious\_match
- matched\_net\_is\_implicit\_supply
- matched\_net\_implicit\_reason

In questo modo il grafo non è solo una struttura topologica, ma anche una rappresentazione diagnostica della qualità del matching.

## 7.4. Net implicite e supply non esplicite

Le net possono essere marcate come:

- is\_implicit\_supply
- implicit\_reason

Questa informazione compare sia nei nodi net sia negli archi `CONNECTED_TO`, così l'utente finale può distinguere connessioni esplicite e alimentazioni inferite.

## 7.5. JSON semplificato per lettura umana

Oltre al `graph_json`, viene costruito anche un **simplified JSON** pensato per essere leggibile:

- overview del diagramma;
- contesto diagnostico;
- componenti con terminali e componenti connessi;
- net con terminali e statement testuali;
- `terminal_facts` in forma discorsiva.

Questo è un output molto utile anche per la scrittura della tesi, perché traduce il risultato tecnico in una forma più narrativa.

## 7.6. CSV per nodi e archi

Il passo 07 esporta anche:

- `nodes.csv`
- `edges.csv`
- CSV combinati batch-wide

Questi file sono utili per analisi tabellari e per eventuali import in graph database.

## 8. Passo 08 — Visualizzazione del grafo

File principale: `08_visualize_graph.py`

Il passo 08 genera una serie di visualizzazioni automatiche del grafo esportato.

### 8.1. Tipi di vista prodotti

Le viste previste sono:

- **full graph**
- **component** → **net**
- **overlay sul diagramma**
- **index.html batch**

### 8.2. Output statici e interattivi

Il sistema produce:

- PNG statiche;
- HTML interattivi;
- dashboard batch finale.

Inoltre copia nelle cartelle `downloads` :

- i graph JSON;
- i simplified JSON.

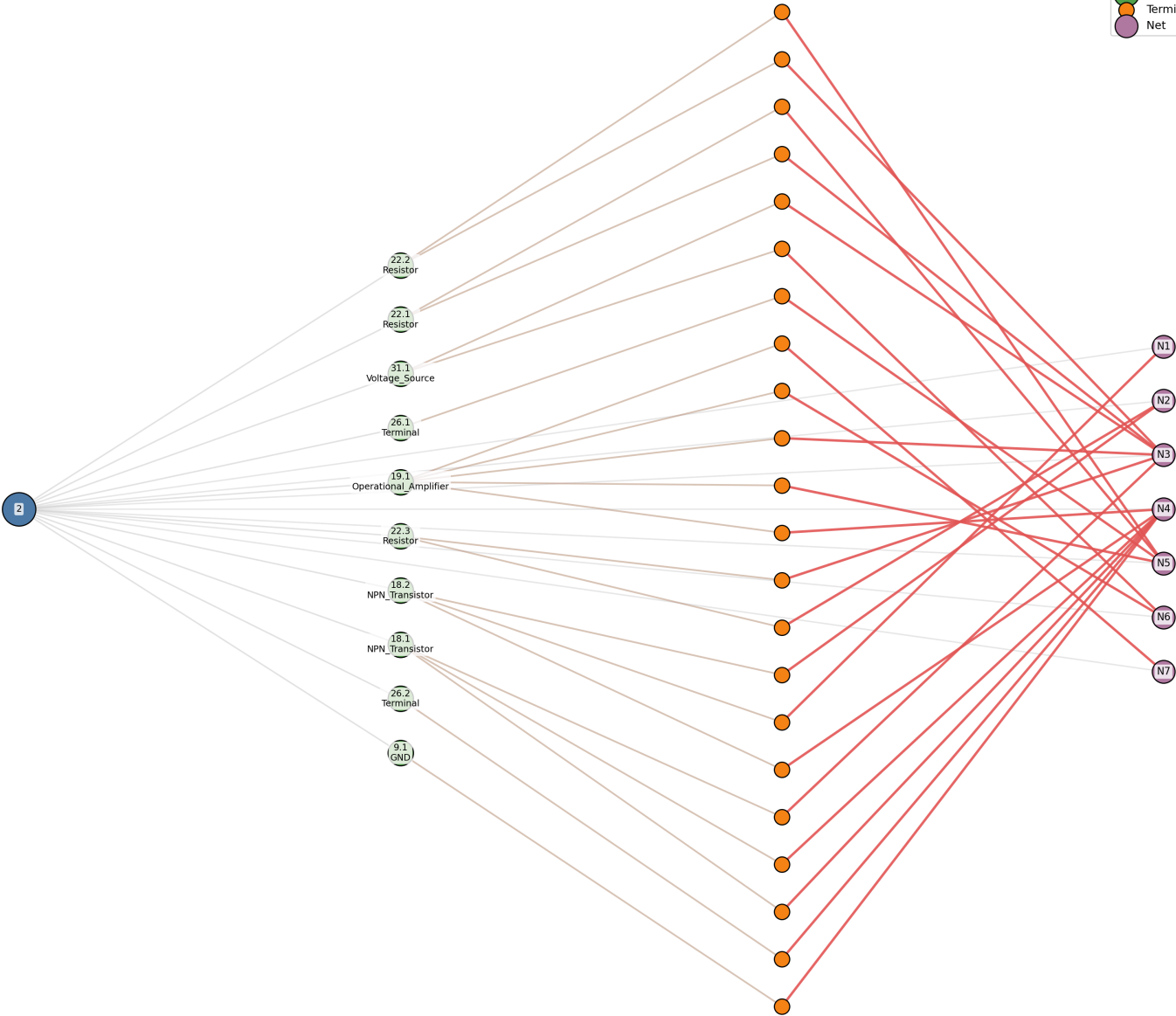
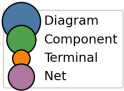
### 8.3. Dashboard finale

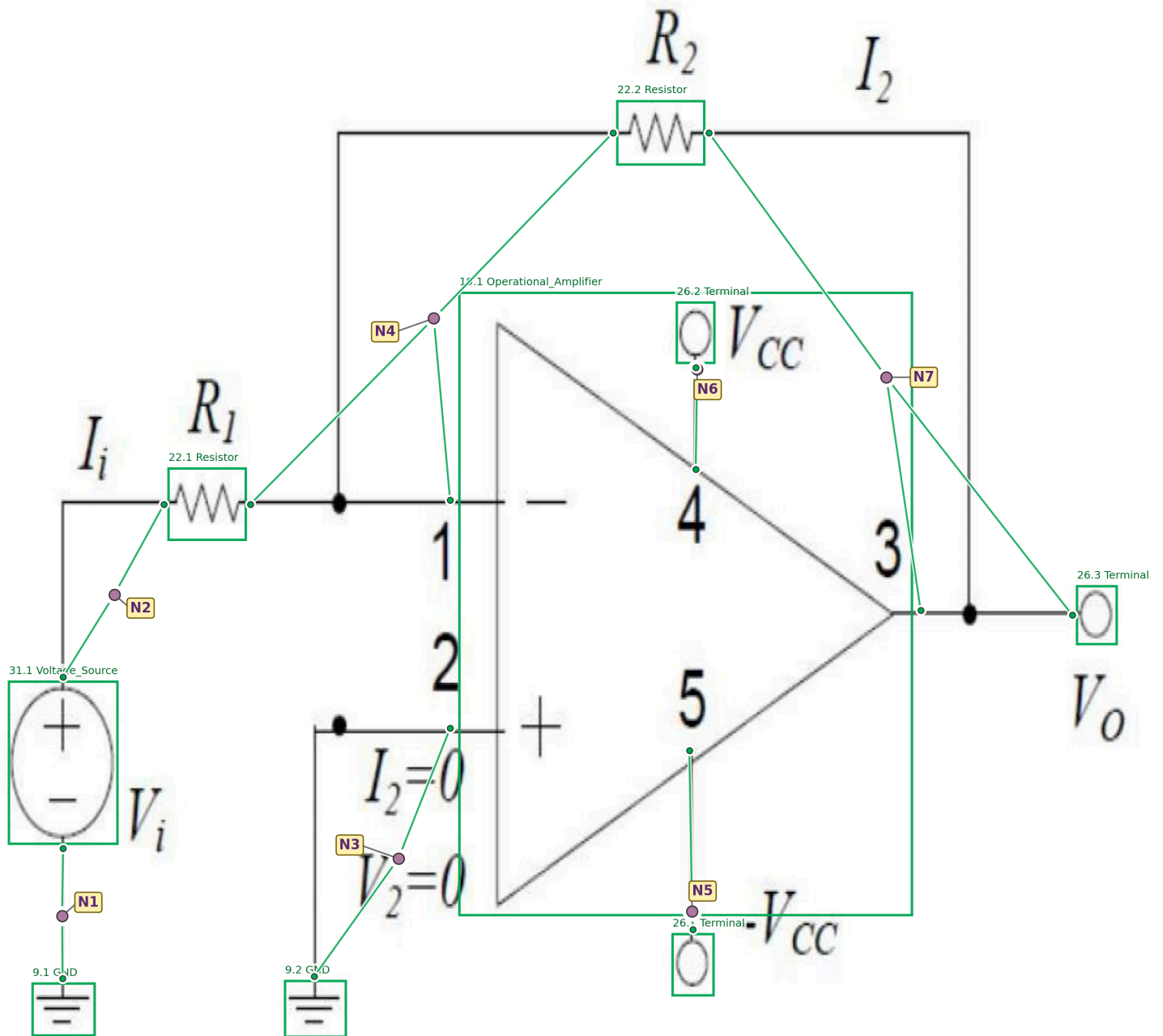
L' `index.html` batch raccoglie, per ogni diagramma:

- numero totale di nodi;
- numero totale di archi;
- numero di terminali sospetti;
- numero di implicit supply nets;
- link alle visualizzazioni e ai JSON.

Questo rende il passo 08 il punto finale di consultazione della pipeline.

Full graph - 2  
nodes=40 | edges=61 | suspicious=0





## 9. Sintesi delle modifiche più importanti

Dal punto di vista metodologico, le modifiche principali introdotte nella pipeline possono essere riassunte così:

### 9.1. Passaggio da regole geometriche rigide a localizzazione evidence-based

Invece di fissare i terminali solo come centri dei lati dei bbox, la pipeline usa ora:

- side peak localization;



- scoring locale e far-field;
- verifica point-based;
- refine su punti strutturali.

## 9.2. Gestione dedicata dei componenti critici

I componenti più problematici sono stati trattati con strategie dedicate:

- `Terminal` : classificazione uno/due terminali molto conservativa;
- `NPN_Transistor` : prefilter d'asse e bonus per il lato base;
- `Mosfet` : validazione finale basata sui terminal points;
- `Operational_Amplifier` : orientazione con soli terminali obbligatori, rilevamento strutturale degli aux e snapping al nearby terminal.

## 9.3. Miglioramento progressivo della robustezza topologica

Nei passi 04–06 la pipeline è diventata molto più robusta grazie a:

- keep zones direzionate sui terminali;
- filtering del rumore prima dello skeleton;
- matching delle net con preferenze dal passo precedente;
- gestione delle implicit supply;
- confidence e warning sui match finali.

## 9.4. Arricchimento semantico dell'output

Nei passi 07–08 il risultato non è più solo una topologia grezza, ma una struttura completa che include:

- grafo esplicito;
- stato del matching;
- net implicite;
- contesto diagnostico;
- viste statiche e interattive;
- JSON semplificati leggibili.